

P0454r0

Wording for a Minimal `mdspan`<>

Draft Proposal, 10 October 2016

This version:

<http://wg21.link/P0454r0>

Authors:

[H. Carter Edwards](#) (Sandia National Laboratory)

[Bryce Adelstein Lelbach](#) (Lawrence Berkeley National Laboratory)

Audience:

LEWG, LWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Proposed wording, based on span [\[P0122R3\]](#) for an `mdspan` with a core subset of the full `mdspan` functionality.

Table of Contents

- 1 **Modified P0122R3 Wording**
- 2 **`mdspan` Wording**
 - 2.1 `dimensions` Class Template
 - 2.2 Layout Mapping Requirements
 - 2.3 `mdspan` Class Template
 - 2.4 Layout Mapping Properties
- 3 **Feature Testing**
 - References**
 - Informative References

§ 1. Modified P0122R3 Wording

The proposed wording changes are relative to the working draft of the standard as of [\[N4567\]](#) with the changes proposed in [\[P0122R3\]](#) applied.

The  character is used to denote a placeholder section number which the editor shall determine.

Apply the following changes to 23.7. [views.general] paragraph 1:

The header `<span>` defines the `view` `views` `span` and `mdspan`. A `span` is a view over a contiguous sequence of objects, the storage of which is owned by some other object. An `mdspan` is a multidimensional view over a contiguous sequence of objects, the storage of which is owned by some other object.

Apply the following changes to 23.7..1 [views.span.synop]:

```
namespace std {
```

```

// [views.constants], constants
constexpr ptrdiff_t dynamic_extent = -1;
enum class dynamic_extent_tag {};
inline constexpr dynamic_extent_tag dyn { -1 };

template <auto Extent>
using is_dynamic_dimension = typename is_same<decay_t<decltype(Extent)>,
dynamic_extent_tag>::type;
template <auto Extent>
inline constexpr bool is_dynamic_dimension_v =
is_dynamic_dimension<Extent>::value;

// [views.span], class template span
template <class ElementType, ptrdiff_t auto Extent = dynamic_extent>
class span;

// [views.span.comparison], span comparison operators
template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator==(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator!=(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator<(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator<=(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator>(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

template <class ElementType, ptrdiff_t auto Extent>
constexpr bool operator>=(const span<ElementType, Extent>& l, const
span<ElementType, Extent>& r);

// [views.span.objectrep], views of object representation
template <class ElementType, ptrdiff_t auto Extent>
span<const char, ((Extent == dynamic_extent) ? dynamic_extent :
(sizeof(ElementType) * Extent))> as_bytes(span<ElementType, Extent> s)
noexcept;

template <class ElementType, ptrdiff_t auto Extent>
span<const char, ((Extent == dynamic_extent) ? dynamic_extent :
(sizeof(ElementType) * Extent))> as_writable_bytes(span<ElementType,
Extent>) noexcept;

// [views.dimensions], class template dimensions
template <auto... Extents>
class dimensions;

// [views.dimensions.comparison], dimensions comparison operators
template <auto... Extents>
constexpr bool operator==(const dimensions& l,
const dimensions& r);

template <auto... Extents>
constexpr bool operator!=(const dimensions& l,
const dimensions& r);

```

```
// [views.mdspan], class template mdspan
template <class ElementType, class Dimensions, class... Properties>
class mdspan;

template <class T> struct is_mdspan_property;
template <class T>
inline constexpr bool is_mdspan_property_v = is_mdspan_property<T>::value;

// [views.layout.props], mdspan layout properties
class layout_right;
class layout_left;

template <class T> struct is_layout;
template <class T>
inline constexpr bool is_layout_v = is_layout<T>::value;

} // namespace std
```

Add the following paragraph in between paragraphs 2 and 3 in 23.7.4.2 [views.span.template]:

`span` requires the type of `Extent` to be either `dynamic_extent_tag` or implicitly convertible to the `span`'s `index_type`.

Apply the following changes to the definition of `span` in 23.7.4.2 [views.span.template]

```
namespace std {

// A view over a contiguous, single-dimension sequence of objects
template <class ElementType, ptrdiff_tauto Extent = dynamic_extentdyn>
class span {

    inline constexpr static index_typeauto extent = Extent;

    template <class OtherElementType, ptrdiff_tauto OtherExtent>
        constexpr span(const span<OtherElementType, OtherExtent>& other);
    template <class OtherElementType, ptrdiff_tauto OtherExtent>
        constexpr span(span<OtherElementType, OtherExtent>&& other);

    template <ptrdiff_t Offset, ptrdiff_tauto Count = dynamic_extentdyn>
        constexpr span<element_type, Count> subspan() const;
    constexpr span<element_type, dynamic_extent> first(index_type count) const;
    constexpr span<element_type, dynamic_extent> last(index_type count) const;
    constexpr span<element_type, dynamic_extent> subspan(index_type
        offset) const;
    constexpr span<element_type, dynamic_extent> subspan(index_type
        offset, index_type count --dynamic_extent) const;
```

Apply the following changes to the converting copy constructor in 23.7.4.3 [views.span.cons]:

```
template <class OtherElementType, ptrdiff_t auto OtherExtent>
constexpr span(const span<OtherElementType, OtherExtent>& other);
template <class OtherElementType, ptrdiff_t auto OtherExtent>
constexpr span(span<OtherElementType, OtherExtent>&& other);
```

Remarks: These constructors shall not participate in overload resolution unless trying to access `OtherElementType` through an `ElementType*` would meet the rules for well-defined object access defined in 3.10/10.

Requires: If `extent is not equal to dynamic_extent` `is_dynamic_dimension_v<Extent> == false`, then `other.size()` shall be equal to `extent`.

Effects: Constructs a `span` by copying the implementation data members of another `span`, performing suitable conversions.

Postconditions: `size() == other.size()` && `data() == reinterpret_cast<pointer>(other.data())`.

Complexity: Constant.

Throws: Nothing.

Apply the following changes to `subspan()` in 23.7.4 [views.span.sub]:

```
template <ptrdiff_t Offset, ptrdiff_t auto Count = dynamic_extent dyn>
constexpr span<element_type, Count> subspan() const;
```

Requires: `(Offset == 0 || Offset > 0 && Offset < size()) && (Count == dynamic_extent || is_dynamic_dimension<Count> || Count >= 0 && Offset + Count <= size())`.

Effects: Returns a new `span` that is a view over `Count` elements of the current span starting at element `Offset`. If `Count is equal to dynamic_extent` `is_dynamic_dimension<Count> == true`, then a `span` over all elements from `Offset` onwards is returned.

Returns: `span(data() + Offset, Count == dynamic_extent || is_dynamic_dimension<Count> ? size() - Offset : Count)`.

Complexity: Constant.

Add the following `subspan()` overload to 23.7.4 [views.span.sub]:

```
constexpr span<element_type, dynamic_extent> subspan(index_type
    offset) const;
```

Requires: `(offset == 0 || offset > 0 && offset < size())`.

Effects: Returns a new `span` that is a view over all elements from `offset` onwards.

Returns: `span(data() + offset, size() - offset)`.

Complexity: Constant.

Apply the following changes to `subspan()` in 23.7.4 [views.span.sub]:

```
constexpr span<element_type, dynamic_extent> subspan(index_type  
offset, index_type count = dynamic_extent) const;
```

Requires: `(offset == 0 || offset > 0 && offset < size()) && (count = dynamic_extent || count
>= 0 && offset + count <= size())`.

Effects: Returns a new `span` that is a view over `count` elements of the current span starting at element `offset`. ~~If count is equal to `dynamic_extent`, then a span over all elements from `offset` onwards is returned.~~

Returns: `span(data() + offset, count = dynamic_extent ? size() - offset + count)`.

Complexity: Constant.

§ 2. `mdspan` Wording

§ 2.1. `dimensions` Class Template

Add the following section, 23.7.❖ [views.dimensions.template]:

23.7.◆ Class Template `dimensions` [`views.dimensions.template`]

```
template <auto... Extents>
class dimensions {
public:
    // types
    using value_type = ptrdiff_t;
    using index_type = ptrdiff_t;

    // [views.dimensions.cons], constructors/assignment/destructor
    constexpr dimensions() noexcept;
    template <class... DynamicExtents>
        constexpr dimensions(DynamicExtents... dexts) noexcept;
    constexpr dimensions(dimensions const& other) noexcept = default;
    constexpr dimensions(dimensions&& other) noexcept = default;
    dimensions& operator=(dimensions const& other) noexcept = default;
    dimensions& operator=(dimensions&& other) noexcept = default;

    // [views.dimensions.obs], observers
    static constexpr index_type rank() noexcept;
    static constexpr index_type rank_dynamic() noexcept;

    constexpr size_type size() noexcept;

    constexpr bool is_dynamic_dimension(index_type i) const noexcept;

    // [views.dimensions.elem], element access
    constexpr value_type operator[](index_type i) const noexcept;
};
```

`dimensions` is a class which contains storage for a fixed number of integer elements, each of which represents the extent of a *dimension*, collectively forming a multi-dimensional integer index.

Each non-type template parameter specifies either a positive integral value, indicating a *static dimension*, or a value of type `dynamic_extent_tag`, indicating a *dynamic dimension* whose value will be provided at runtime.

Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the contained dimensions. The contained values shall be allocated in a region of the `dimensions` storage suitably aligned for the type `value_type`. [*Note*: Implementations are not required to store the value of static dimensions. — *end note*]

Add the following section, 23.7.◆ [`views.dimensions.cons`]:

23.7.❖ `dimensions` Constructors, Assignment and Destructors [views.dimensions.cons]

```
constexpr dimensions() noexcept;
```

Effects: Constructs a `dimensions` object with all dynamic dimensions default initialized.

```
template <class... DynamicExtents>
constexpr dimensions(DynamicExtents... dexts) noexcept;
```

Remarks: This constructor shall not participate in overload resolution unless `sizeof...(DynamicExtents) == rank_dynamic()`.

Effects: Constructs a `dimensions` object with each dynamic dimensions initialized from a corresponding value in `dexts`.

```
constexpr dimensions(dimensions const& other) noexcept = default;
constexpr dimensions(dimensions&& other) noexcept = default;
```

Effects: Constructs a `dimensions` object by copying from another `dimensions` object.

```
dimensions& operator=(dimensions const& other) noexcept = default;
dimensions& operator=(dimensions&& other) noexcept = default;
```

Effects: Assigns from one `dimensions` object into another.

Add the following section, 23.7.❖ [views.dimensions.obs]:

23.7.❖ `dimensions` Observers [views.dimensions.obs]

```
static constexpr index_type rank() noexcept;
```

Effects: Returns the number of elements in the `dimensions` object.

Returns: `sizeof...(Extents)`.

```
static constexpr index_type rank() noexcept;
```

Effects: Returns the number of parameters in `Extents` for which `is_dynamic_dimension_v` is true.

```
constexpr bool is_dynamic_dimension(index_type i) const noexcept;
```

Effects: If `i < rank()`, returns true if the `i`th dimension is a dynamic dimension. Otherwise, returns false.

Add the following section, 23.7.❖ [views.dimensions.elem]:

23.7.❖ `dimensions` Element Access [views.dimensions.elem]

```
constexpr value_type operator[](index_type i) const noexcept;
```

Effects: If `i < rank()`, returns the value of the `i`th element. Otherwise, returns 0.

Add the following section, 23.7.❖ [views.dimensions.comparison]:

23.7. `dimensions` Comparison Operators [views.dimensions.comparison]

```
template <auto... Extents>
constexpr bool operator==(const dimensions& l,
                          const dimensions& r);
```

Effects: `l[i] == r[i]`, where `i` is in the range `[0, rank())`.

Throws: Nothing.

```
template <auto... Extents>
constexpr bool operator!=(const dimensions& l,
                          const dimensions& r);
```

Effects: Equivalent to `return !(l == r);`

§ 2.2. Layout Mapping Requirements

Add the following section, 23.7.  [views.layout.requirements]:

23.7. Layout Mapping Requirements [views.layout.requirements]

A *layout* is class that describes a mapping from a multi-dimensional index to a *mapped index*, which is one-dimensional. `mdspan` (23.7. ) is parameterized in terms of layout mappings.

A layout shall have an embedded template class `mapping` which takes a single template parameter that shall be a type which fulfills the definition of the `dimensions` class. *Layout mappings* are instantiations of this embedded template class.

In the following section:

- Let `LM` be a layout mapping and let `lm` be an object of type `LM`.
- Let `i` be in the range `[0, rank())` in order.

The expression `is_layout<LM>::value` shall be true.

The expression `lm.span()` shall be well-formed and have the following semantics:

- *Effects:* Returns `(*this)(extent(0), /* ... */, extent(i)) - (*this)(0, /* ... */ , 0)`.

The expression `LM::is_always_strided(r)` shall be true if the difference between two mapped indices that are consecutive is always identical.

If `LM::is_always_strided(r)`, the expression `lm.stride(r)` shall be well-formed and have the following semantics:

- *Effects:* Returns the difference between two mapped indices that are consecutive.

The expression `lm(is...)` shall be well-formed and have the following semantics if `sizeof...(decltype(is)) == LM::rank()` and all the types in `decltype(is)` are integral:

- *Effects:* Returns the mapped index for multi-dimensional index `is`.

§ 2.3. `mdspan` Class Template

Add the following section, 23.7.  [views.mdspan.template]:

23.7. Class template `mdspan` [views.mdspan.template]


```

template <class ElementType, class Dimensions, class... Properties>
class mdspan {
public:
    // types
    using element_type = ElementType;
    using index_type = ptrdiff_t;
    using difference_type = ptrdiff_t;
    using pointer = element_type*;
    using reference = element_type&;

    using layout = implementation defined;
    using layout_mapping = typename layout::template mapping;

    // [views.mdspan.cons], constructors/assignment/destructor
    constexpr mdspan() noexcept;
    constexpr mdspan(nullptr_t) noexcept;
    template <class... DynamicExtents>
        constexpr mdspan(pointer ptr, DynamicExtents... dexts);
    constexpr mdspan(pointer ptr, layout_mapping&& l);
    constexpr mdspan(pointer ptr, layout_mapping const& l);
    constexpr mdspan(mdspan const& other) noexcept = default;
    constexpr mdspan(mdspan&& other) noexcept = default;
    template <class UElementType, class UDimensions, class... UProperties>
        constexpr mdspan(mdspan<UElementType, UDimensions, UProperties...> const& other);
    template <class UElementType, class UDimensions, class... UProperties>
        constexpr mdspan(mdspan<UElementType, UDimensions, UProperties...>&& other);

    mdspan& operator=(mdspan const& other) noexcept = default;
    mdspan& operator=(mdspan&& other) noexcept = default;
    template <class UElementType, class UDimensions, class... UProperties>
        mdspan& operator=(mdspan<UElementType, UDimensions, UProperties...> const& other);
    template <class UElementType, class UDimensions, class... UProperties>
        mdspan& operator=(mdspan<UElementType, UDimensions, UProperties...>&& other);

    ~mdspan() noexcept = default;

    // [views.mdspan.domobs], domain observers
    static constexpr index_type rank() noexcept;
    static constexpr index_type rank_dynamic() noexcept;

    constexpr bool is_dynamic_dimension(rank_type r) const noexcept;
    constexpr index_type extent(rank_type r) const noexcept;

    constexpr index_type size() const noexcept;
    constexpr index_type length() const noexcept;
    constexpr bool empty() const noexcept;

    // [views.mdspan.codobs], codomain observers
    constexpr index_type span() const noexcept;

    // [views.mdspan.mapobs], mapping observers
    constexpr index_type stride(rank_type r) const noexcept;

    constexpr layout_mapping mapping() const noexcept;

    // [views.mdspan.elem], element access
    template <class... Indices>
        reference operator()(Indices... is) const;

    constexpr pointer data() const noexcept;
};

```

1. An `mdspan` is a multidimensional view over a contiguous sequence of objects, the storage of which is owned by some other object.

2. `ElementType` is required to be a complete object that is not an abstract class type.
3. In the following section:
 - Let i be in the range $[0, \text{sizeof}...(Properties))$ in order and
 - P^i be the i th type in `Properties`,
 - Let r be in the range $[0, \text{rank}())$ in order and
 - Idx^r be the r th type in a template parameter pack named `Indices`,
 - Let rd be in the range $[0, \text{rank_dynamic}())$ in order, and
 - $DExt^{rd}$ be the rd th type in a template parameter pack named `DynamicExtents`.
4. `is_layout_v<Pi>` shall be true for i . The `layout` type is the type in P^i for which `is_layout_v<Pi> == true`. If `sizeof...(Properties) == 0` or `is_layout_v<Pi> == false` for all i , then the `layout` type is `layout_right`.
5. If `is_mdspan_property_v<Pi>` is false for any i , then `mdspan` is ill-formed.

Add the following section, 23.7.◆ [views.mdspan.cons]:

23.7.◆ `mdspan` Constructors, Assignment and Destructors [views.mdspan.cons]

```
constexpr mdspan() noexcept;
constexpr mdspan(nullptr_t) noexcept;
```

Remarks: If `!is_dynamic_dimension(r) && extent(r) != 0` then the program is ill-formed.

Effects: Constructs an empty `mdspan`.

Postconditions: `size() == 0 && data() == nullptr`

```
template <class... DynamicExtents>
constexpr mdspan(pointer ptr, DynamicExtents... dexts);
```

Effects: Equivalent to `mdspan(ptr, Dimensions(dexts...))`.

Complexity: Constant.

```
constexpr mdspan(pointer ptr, layout_mapping&& l);
constexpr mdspan(pointer ptr, layout_mapping const& l);
```

Requires:

- `l.size() >= 0`.
- If `ptr` is null, then `l.size()` shall be 0.
- If `ptr` is not null, `ptr` shall point to the beginning of a valid sequence of objects of at least `l.size()` length.

Effects: Constructs an `mdspan` that is a view over the sequence of objects pointed to be `ptr` with mapping `l`. If `ptr` is null or `l.size()` is 0, then an empty `mdspan` is constructed.

Complexity: Constant.

Throws: Nothing.

```
constexpr mdspan(mdspan const& other) noexcept = default;
constexpr mdspan(mdspan&& other) noexcept = default;
```

Effects: Constructs an `mdspan` by copying from another `mdspan`.

Postconditions: `other.size() == size() && other.data() == data() && other.mapping() == mapping()`

```
template <class UElementType, class UDimensions, class... UProperties>
constexpr mdspan(mdspan<UElementType, UDimensions, UProperties...> const& other);
template <class UElementType, class UDimensions, class... UProperties>
constexpr mdspan(mdspan<UElementType, UDimensions, UProperties...>&& other);
```

Remarks: This constructor shall not participate in overload resolution unless `mdspan<UElementType, UDimensions, UProperties...>::pointer` is implicitly convertible to `pointer`.

Requires: If `!other.is_dynamic_dimension(r)` then `other.extent(r)` shall be equal to `extent(r)`, for all `r`.

Effects: Constructs an `mdspan` by copying from another `mdspan`.

Postconditions: `other.size() == size() && reinterpret_cast<pointer>(other.data()) == data() && other.mapping() == mapping()`

Complexity: Constant.

Throws: Nothing.

```
mdspan& operator=(mdspan const& other) noexcept = default;
mdspan& operator=(mdspan&& other) noexcept = default;
```

Effects: Assigns from one `mdspan` to another.

Postconditions: `other.size() == size() && other.data() == data() && other.mapping() == mapping()`

```
template <class UElementType, class UDimensions, class... UProperties>
mdspan& operator=(mdspan<UElementType, UDimensions, UProperties...> const& other);
template <class UElementType, class UDimensions, class... UProperties>
mdspan& operator=(mdspan<UElementType, UDimensions, UProperties...>&& other);
```

Remarks: This operator shall not participate in overload resolution unless `mdspan<UElementType, UDimensions, UProperties...>::pointer` is implicitly convertible to `pointer`.

Requires: If `!other.is_dynamic_dimension(r)` then `other.extent(r)` shall be equal to `extent(r)`, for all `r`.

Effects: Assigns from one `mdspan` to another.

Postconditions: `other.size() == size() && other.data() == data() && other.mapping() == mapping()`

Add the following section, 23.7.❖ [views.mdspan.domobs]:

23.7.◆ `mdspan` Domain Observers [`views.mdspan.domobs`]

```
static constexpr index_type rank() noexcept;
```

Effects: Equivalent to `return layout_mapping::rank();`

```
static constexpr index_type rank_dynamic() noexcept;
```

Effects: Equivalent to `return layout_mapping::rank_dynamic();`

```
constexpr bool is_dynamic_dimension(rank_type r) const noexcept;
```

Effects: Equivalent to `return mapping().is_dynamic_dimension(r);`

```
constexpr index_type extent(rank_type r) const noexcept;
```

Effects: Equivalent to `return mapping()[r];`

```
constexpr index_type size() const noexcept;  
constexpr index_type length() const noexcept;
```

Effects: Equivalent to `return mapping().size();`

```
constexpr bool empty() const noexcept;
```

Effects: Equivalent to `return size() == 0;`

Add the following section, 23.7.◆ [`views.mdspan.codobs`]:

23.7.◆ `mdspan` Codomain Observers [`views.mdspan.codobs`]

```
constexpr index_type span() const noexcept;
```

Effects: Equivalent to `return mapping().span();`

Add the following section, 23.7.◆ [`views.mdspan.mapobs`]:

23.7.◆ `mdspan` Mapping Observers [`views.mdspan.mapobs`]

```
constexpr index_type stride(rank_type r) const noexcept;
```

Remarks: This function shall not participate in overload resolution if `layout_mapping::is_always_strided(r)` is true for all `r`.

Effects: Equivalent to `return mapping().stride(r);`

```
constexpr layout_mapping mapping() const noexcept;
```

Effects: Returns a copy of the `mdspan`'s layout mapping.

Add the following section, 23.7.◆ [`views.mdspan.mapobs`]:

23.7.◆ `mdspan` Element Access [views.mdspan.elem]

```
template <class... Indices>
    reference operator()(Indices... is) const;
```

Effects: Returns a reference to the element at position `mapping()(is...)`.

Complexity: Constant

Throws: Nothing

```
constexpr pointer data() const noexcept;
```

Effects: If `!empty()`, returns a pointer to the first element in the sequence accessible via the `mdspan`. Otherwise, returns `nullptr`.

Complexity: Constant

§ 2.4. Layout Mapping Properties

Add the following section, 23.7.◆ [views.layout.props]:

23.7.◆ Layout Mapping Properties [views.layout.props]

In the following section, let `D` be an instantiation of the `dimensions` class template.

`layout_right` shall be a layout ([views.layout.requirements]) such that `layout_right::mapping<D>().stride(D::rank() - 1) == 1`.

`layout_left` shall be a layout such that `layout_left::mapping<D>().stride(0) == 1`.

§ 3. Feature Testing

The `__cpp_lib_mdspan` feature test macro should be added.

§ References

§ Informative References

[N4567]

Richard Smith. Working Draft, Standard for Programming Language C++ Note. 9 November 2015. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf>

[P0122R3]

Neil MacIntosh. span: bounds-safe views for sequences of objects. 10 July 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0122r3.pdf>